Küresel Liderlik Hedefli Ultra Performanslı Git ve CI/CD Platform Mimari Tasarımı

Bu doküman; Forgejo, Woodpecker, GitHub ve GitLab'e alternatif olacak; **küresel ölçekte liderlik iddiası taşıyan, ultra performanslı, C/C++ ve GNU Assembler (GAS) odaklı** bir Git ve CI/CD platformunun mimari yol haritasını içerir.

---

1. Çekirdek (Core) Mimari ve Dil Dağılımı

Sistemde en yüksek performansı elde etmek için görevleri dillere göre doğru paylaştırmak gerekir.

- **GNU Assembler (GAS):** Sadece işlemciye (CPU) doğrudan bağımlı olan, bellek kopyalama (zero-copy), şifreleme/çözme algoritmaları (SHA-256, SSH anahtar doğrulamaları) ve Git paket (packfile) ayrıştırma döngülerinin en kritik **hot-path** (en sık çalışan) kısımlarında kullanılmalıdır.
- **C / C++ (C++20/23):** Ana Git sunucusu motoru (Git akıllı protokol yönetimi - `git-upload-pack`, `git-receive-pack`), dosya sistemi g/ç (I/O) işlemleri, bellek yönetimi (Custom Memory Pool kullanan yapılar) ve gRPC tabanlı iç servis haberleşmeleri için temel taş olmalıdır.
- **Go (Golang):** Web backend'in API yönlendirme (routing), web sunucusu yönetimi, kullanıcı yetkilendirme (JWT/OAuth) ve PostgreSQL/Redis bağlantı havuzlarını yöneten üst katmanı için idealdir. Go'nun eşzamanlılık (concurrency - goroutines) yeteneği, ağ isteklerini C++ çekirdeğine en az maliyetle aktarmak için köprü görevi görür.

---

2. Veri Yönetimi ve Depolama Katmanı

- **Veritabanı (PostgreSQL):** Kullanıcı verileri, repository metadata'ları (yıldızlar, yorumlar, pull request durumları) ve CI/CD loglarının indeksleri için **PostgreSQL** kullanılmalıdır. Ölçeklenebilirlik için **TimescaleDB** (CI/CD metrikleri için) ve **Citus** (sharding/yatay ölçekleme için) eklentileri değerlendirilmelidir.
- **Önbellek & Kuyruk (Redis):** SSH anahtarı doğrulamaları, oturum yönetimi, web sunucu önbelleği ve anlık bildirimler (WebSockets) için **Redis** (veya C++ ile yazılmış ultra hızlı bir alternatifi olan **Dragonfly**) konumlandırılmalıdır.

- **Git Objeleri & Paket Depolama:** Ham Git verileri yerel NVMe disklerde Epoll tabanlı C++ servisleriyle işlenmeli, uzun vadeli arşivleme için S3 uyumlu nesne depolama (Object Storage) sistemlerine asenkron olarak aktarılmalıdır.

⚡ 3. Ultra Performanslı CI/CD Motoru (Woodpecker Alternatifi)

Geleneksel CI/CD sistemleri (Runner'lar) ajan tabanlıdır ve her iş için ağır Docker container'ları ayağa kaldırır. Küresel bir liderlik için buradaki mimariyi kökten değiştirmelisiniz:

- **Hafif Sanallaştırma:** Docker yerine Linux çekirdeğinin (Kernel) sağladığı **Namespaces**, **Cgroups v2** ve **seccomp** özelliklerini doğrudan C++ ile kontrol eden, "Micro-Sandbox" veya Linux **KVM tabanlı Firecracker** benzeri ultra hafif, milisaniyeler içinde açılan izole çalışma ortamları kurun.
- **Derleme Önbelleği (Build Caching):** C/C++ odaklı olacağınız için yerleşik bir global `ccache` benzeri dağıtık derleme önbellek mekanizmasını mimariye entegre edin.

🌐 4. Web Backend ve Frontend Teknolojileri

- **Backend:** Go tarafında **Fiber** veya **Gin** gibi yüksek performanslı minimalist web framework'leri kullanılabilir. Ancak en yüksek performans için Go'nun standart `net/http` paketi optimize edilmeli veya C++ tarafında **uWebSockets** / **drogon** gibi kütüphanelerle doğrudan HTTP API beslenmelidir.
- **Frontend:** Platformun arayüzünün de backend kadar hızlı açılması gerekir (Jamstack / SSR felsefesi).
  - **Svelte / SvelteKit:** Minimalist yapısı, sanal DOM (Virtual DOM) kullanmaması ve kodu derleme aşamasında saf JavaScript'e dönüştürmesi sayesinde en yüksek arayüz performansını verir.
  - **HTMX + Tailwind CSS:** Eğer SPA (Single Page Application) mimarisinin karmaşıklığından kaçınmak ve backend (Go) tarafından doğrudan render edilmiş süper hızlı HTML parçacıkları dönmek isterseniz, HTMX en iyi modern alternatiftir.

---

5. İzleme, Metrik ve Küresel Dağıtık Ağ (Scale)

- **Zabbix:** Sunucu donanımlarının sağlığı, disk I/O limitleri, NVMe ömürleri ve ağ anahtarlarının (switch) takibi gibi altyapı katmanı (Bare-metal) izlemesinde kullanılmalıdır.
- **Prometheus & Grafana:** C++ servislerinin içerisine gömülecek `prometheus-cpp` kütüphanesi sayesinde uygulamaya özel metrikler (anlık Git klonlama sayısı, saniyedeki paket işleme hızı vb.) toplanmalı ve Grafana ile görselleştirilmelidir.
- **Küresel Ağ Tasarımı (Edge & Anycast):** Dünyanın farklı noktalarında (Amerika, Avrupa, Asya) POP (Point of Presence) noktaları kurulmalı, **BGP Anycast** yönlendirmesiyle kullanıcılar en yakın sunucuya aktarılmalıdır. Git okuma (`git clone`/`fetch`) istekleri akıllı CDN mantığıyla uç sunucularda önbelleğe alınmalıdır.

Doğrudan Uygulanabilir Yol Haritası ve Mimari Özet

Katman	Tercih Edilen Teknoloji	Görevi / Amacı
Kritik İşlemci Döngüleri	GNU Assembler (GAS)	SHA/Kriptografi, Zero-copy bellek yönetimi
Çekirdek Motor (Core Engine)	C++23 / C	Git akıllı protokolü, File I/O, Epoll Ağ Ağları
Web API / Köprü	Golang (Go)	İstek yönlendirme, Eşzamanlılık yönetimi, DB havuzu
Veritabanı	PostgreSQL + Citus	İlişkisel veri, Sharding, Global kullanıcı verisi
Hızlı Depolama / Kuyruk	Redis / Dragonfly	Session, Cache, Anlık CI/CD mesaj kuyrukları
Frontend	Svelte / HTMX + Tailwind	Sıfır gecikmeli, JS yükü hafif kullanıcı arayüzü
Gözlemlenebilirlik	Prometheus + Zabbix + Grafana	Altyapı ve uygulama metriklerinin uçtan uca takibi